Practical and Game-Theoretic Analysis of Path Selection to Defend Against Entry-Exit Correlation in the Tor Network

Aidan E. S.

Dissertation



Swansea University
Prifysgol Abertawe

Department of Computer Science

Adran Gyfrifidureg

28th April 2023

Declaration

Statement 1

This work has not been previously accepted in substance for any degree and is not being concurrently submitted in candidature for any degree.

Signed ..... Aidan E. S.

Date .....

Statement 2

This thesis is the result of my own investigations, except where otherwise stated. Other sources are acknowledged by citations giving explicit references. A bibliography is appended.

Signed ..... Aidan E. S.

Date .....

Statement 3

The University's ethical procedures have been followed and, where appropriate, ethical approval has been granted.

Signed ..... Aidan E. S.

Date .....

Contents

1 Introduction **3**

2 Context and Background **3**
2.1 Tor's Position in History and Society 3
2.2 An Introduction to Tor and Onion Routing 5
2.2.1 Onion Services[37] 6
2.2.2 Path Selection 7
2.2.3 Notable Deviations from the Basic Operation 7

3 Motivation and Goals **8**
3.1 Motivation 8
3.2 Overview of Goals 9

4 Related Work **9**

5 Game Theoretic Analysis **12**
5.1 Introduction 12
5.2 Modelling Our System and Developing Framework 12
5.2.1 Tor Network 12
5.2.2 Adversary 14
5.2.3 Tor Network Designers (Defender) 15
5.3 Modelling Defender Strategies 16
5.3.1 Defender's Strategy to Replicate gPath[44] 16
5.3.2 Defender's Strategy to Replicate Modern Tor 16

6 Implementation **18**
6.1 Analysis Framework 18
6.2 Program Design 18

7 Interactions **19**
7.1 Analysing gPath 19
7.2 Analysing Modern Tor (The Guard System) 20

8 Evaluation **21**
8.1 Implementation 21
8.2 Processes and Project Overall 22

A Addendum, 2026 **23**

1 Introduction

2 Context and Background

2.1 Tor's Position in History and Society

Tor (The Onion Router)[1] saw initial development in the mid 1990s at the United States' Naval Research Laboratory (NRL)[2] - with additional funding from DARPA (Defense Advanced Research Projects Agency). Ostensibly for the purpose of enabling communications with US government employees[3], for example overseas intelligence operatives. The principal persons in this work were David Goldschlag, Michael Reed, and Paul Syverson, all of the Center For High Assurance Computer Systems at the NRL.

By 1996, an initial proof-of-concept with several simulated nodes had been developed for a generation 0 design[1] and ideas were shared in a conference paper[4] at the Workshop on Information Hiding. Work begins on an enhanced generation 1 and continues through 1997. In 1998 several wide-area prototype networks are operational[2] and the independently developed Freedom Network becomes available[2].

Development stalled in 1999 for lack of funding; though resumes once more two years later with generation 1 under the Fault Tolerant Networks Program. In 2002 it is decided that the existing codebase is outdated and has too great a technical debt and thus should be abandoned. Work begins on a new generation 2 design, surprisingly the starting point for this was code that a Cambridge undergraduate had written for a project in their final year[3][2]. This generation is the basis for the Tor network as it operates today; the code was open-sourced in 2004 and by the end of the year 100 nodes were in operation around the world. In 2006 a non-profit, The Tor Project, was founded to oversee development and administration of the network[8].

In 2012, within the decade following the initial deployment of a network resembling the one still used today, a presentation was being given within the very intelligence community that brought about Tor's creation entitled 'Tor Stinks' and theorising on ways the anonymity of the network could be compromised by the NSA and GCHQ[9]. This was one of many documents revealed by Edward Snowden using none other than Tor to communicate his disclosures with media organisations[10]. The coveted goal of one part of government was the sworn enemy of another. Public availability and even popularity was, however, a necessity lest identifying users as those with relations to the US government would be trivialised by the very fact they were accessing the network. A moderate userbase is required for any potential targets of analysis to conceal themselves amongst - this is known as the anonymity set.

Despite the apparently confrontational relationship between authorities and the network, the US government remains the largest financial contributor to the Tor Project[11]. Whether that is a result of altruism, bureaucracy, continuing use for its own intelligence operations, as a surveillance target, or some combination remains speculative[4]. The United States military is

[1] Incidentally, distribution of the source code required a rewrite to modularise cryptographic components due to the export restrictions present in the United States. The same year commercial encryption technology was reclassified under the Commerce Control List rather than the Munitions List.

[2] The Freedom Network[5] (not to be confused with several modern activist groups of the same name) is a defunct corporate effort to create an anonymising network that was active until October 2021[6]. Conceptually the design was extremely similar to Tor with routes of three servers and layers of encryption for each server.

[3] The Student was Matej Pfajfar who at the time of writing is the Chief Technical Officer at the bank Monzo[7]

[4] Whilst it is by no means likely that there is a conspiracy to keep a fundamentally compromised network in

inseparable from the history of Tor but there was never any element of secrecy or deceit - from the beginning, research and code were openly published and users were welcomed to connect to test networks. Nonetheless these varying levels of past and present involvement continue to be a source of suspicion and mistrust from some individuals[14].

It would be amiss for us to fail to make note of Tor's use for criminality. Researchers that obtained a sample of 2419 onion services on Tor with a crawler determined, using their 'Tor-use Motivation Model', that 2.19% were related to child exploitation and 8.8% to illegal drug use[15]. Similarly conducted research[16] produces similar results. There are some caveats to this. Whilst the methodology of this research is not eminently unreasonable, there are 32^{56} possible identifiers for onion sites(compared to, for perspective, 2^{32} addresses in the IPv4 space). It is impossible to say how representative a sample may be. Additionally sites of a certain nature existing in a certain proportion is not a good metric for usage of the network and what proportion of Tor users are navigating to criminal content - an anonymising network is inherently impenetrable to many attempts at analysis. The overriding point, however, is that illegal content and activities are certainly uncomfortably prevalent on the network. The level of cost and risk a society is willing to accept for a tool such as Tor existing is a matter of debate. One certainty is that operations on Tor are by no means impervious to law enforcement. Reports of illegal onion services being taken down and arrests being made are not infrequent[17, 18, 19]. It is by no means true that authorities have proved powerless in this area, effectively used techniques range from exploiting web browser vulnerabilities[20] to more traditional undercover operations[21] and even an attack involving nodes of the network being controlled. This is suspected, but by no means confirmed, to be similar to that being discussed in this project[22, 23].

It is worthwhile to give thought to the vastly different technical landscape in which development was occurring. In 1999 the FCC determined that 'broadband' should be defined as 200 Kbps[24], since 2015 that definition has been 25 Mbps with discussions of increasing it to 100 Mbps[25]. Entirely new means of media consumption have flourished. Additionally the TLS protocol that provides the basis for HTTPS was only defined in 1999[26] and has only more recently achieved near universal adoption on popular websites. Before this development the content of many communications was available to very low-skilled analysis including those sharing a local network with the user. This is the environment Tor was conceived in but it continues to rebut increasing sophisticated attacks and see increasingly widespread use and acceptance[27]. Many major news organizations such as the BBC[28], The Guardian[29] and The New York Times[30] have dedicated onion services[5] using SecureDrop[31] software to enable anonymous communication between journalists and sources. Other sites like Twitter[32] and Facebook[33] provide onion services that mirror the functionality of their regular websites. During 2022 in the midst of the Russian invasion of Ukraine, the CIA invited Russian citizens to establish communications with an onion service[34] and Tor has proven itself to be an exceptionally resilient option for communication in the face of determined censorship attempts during protests in Iran[35] and also for use in China[36].

Anonymity is a luxury most of us can live without for we shall not face persecution for going about our normal lives, reading about what interests us and writing what we believe. That is not the case for all users of the internet and it is only by building safeguarding tools like Tor for

use, nor is it entirely outlandish. First revealed in the same year that work began on onion routing was a decades long operation to distribute compromised hardware encryption devices to over 100 nation states by German and American intelligence[12, 13].

[5] A website that is accessible only via Tor and keeps both user and server anonymous.

them that we can ensure there shall be means of resistance should the day come that we too require them.

2.2 An Introduction to Tor and Onion Routing

Fundamentally onion routing confronts the problem that either the destination of traffic (e.g. the site being browsed) or some party able to observe data moving between source and destination (e.g. an Internet Service Provider) may be malicious and eliminates the need for either to be trusted.

Simply introducing an intermediary through which communications are passed should prevent an observer at the source from determining where the traffic is being sent and likewise for an observer at the destination from determining where the data originated from. However we are unprotected by the intermediate party who we may not be able to trust to be aligned with out interests or to avoid being compromised - the point of failure has shifted. This is the situation present in consumer VPNs and proxy services.

With the provision of three independent nodes rather than a single intermediary, the first node is not aware of the destination, the last node is not aware of the source and neither the first nor last nodes are aware of one another. For a link to be drawn between a user and their activity, all three of the nodes must be compromised which would allow for the entire communication to be traced unless a message seen at an entrance node can be linked to the corresponding one at an exit.

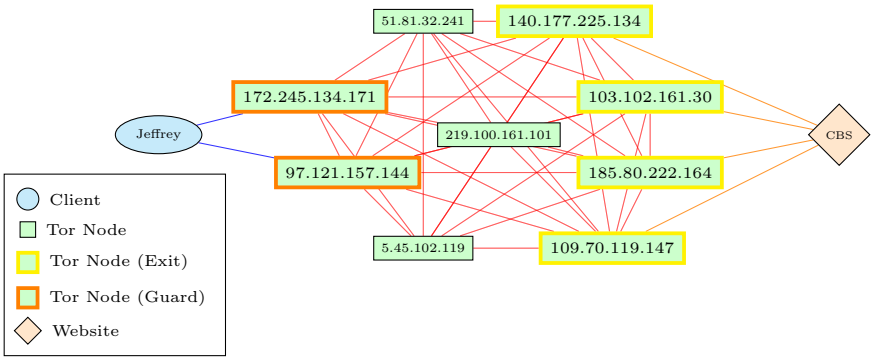


Figure 1: Diagram showing simplified Tor networking.

Tor's namesake of onion routing comes from a metaphor of an onion's layers being applied to layers of encryption on packets passing through the network. A message will first be encrypted with the key of the final node, then that encrypted with the key of the middle node and finally with the key of the first node which strictly enforces the selected route through the network, minimises the information revealed to each node and since a layer of encryption is removed at each hop makes linking a packet that enters a node to the corresponding packet leaving that node difficult.

Entry-Exit linking reduces the resources that must be expended to conduct an attack as only 2, rather than the full 3, nodes selected for a path must be under the control of an adversary. This requires matching a packet sent by a node that is first in a path to that received by a node that is last in the path despite middle node's layer of encryption. The practicalities of this attack are looked at in section 4, Related Work.

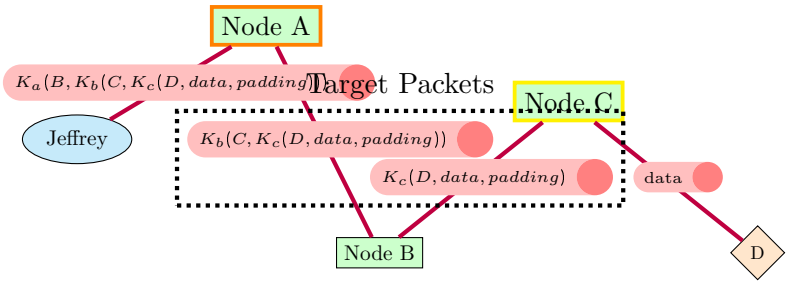


Figure 2: Simplification of a typical Tor packet originating from a client - where $K_x()$ is data encrypted using the public key for X; A, B, C describe the locations of respective Tor nodes such that data may be routed to them and D the location of the user's final destination. Included with the user's data is some padding lest it would be trivial to perform correlation with packet size alone. Each relay removes its own layer of encryption to discover a packet of data and information on where it should be sent.

2.2.1 Onion Services[37]

Onion services, formerly called hidden services but it appears that was considered to have negative connotations, allow websites to be run that are accessible exclusively via the Tor network and obscure the identity of the operator similar to the protections for clients. We will very briefly look at their operation as there are some insights for Entry-Exit linking.

An onion service has a private key and a public key from which its address is obtained - in version 3 of the system that is a 56 character base32 encoded string of a 32 byte key, a version number and checksum[38]. For example, the address for the BBC's international site is `bbcweb3hytmzhn5d532owbu6oqadra5z3ar726vq5kgwwn6aucdccrad.onion`. The customised first few characters are achieved just by brute force generation.

The service will select several nodes to act as introduction points and connects to these via two intermediary nodes for a circuit of three. The identities of the introduction points are signed and added to a distributed hash table shared amongst nodes acting as Hidden Service Directories. A client will form a path to a directory, lookup the address of the desired service and select one of the introduction points to connect with via three intermediaries. The client will then select yet another node as a rendezvous point, connect via two intermediaries and communicate this to the service. The service will now also connect to the rendezvous point though via 3 intermediaries and finally the client is able to access the service. As you may expect this results in very significant latency and initial connection in particular is quite slow. Notice that whenever the client and service are communicating it is via a total of six Tor nodes, three of which have been selected by each party.

If an adversary has a particular onion service they wish to deanonymise, the obvious starting point is the introduction points. If a lookup reveals that a compromised node is already in use then the adversary has a good starting point for an entry-exit linking attack. If not then a motivated adversary may wish to attack the current introduction points forcing the service to select new ones lest they become inaccessible which gives the adversary another chance to have one of their nodes selected. This is practically an inverse of the problems with guard nodes. Similarly this also allows for effective denial of service attacks on onion services which is a severe limitation of the design for avoiding censorship but this is an area the Tor Project is looking to work on.[39, 40].

2.2.2 Path Selection

There are nine Directory Authorities spread across six countries with responsibility for relays which individually compile information of the state on the network and communicate on an hourly basis to form a document called the Consensus that is signed by each authority. The identities of the directory authorities and their public keys is hardcoded in each Tor client. With the ability to verify the signatures the client can obtain from any single authority a copy of the Consensus that will contain all necessary information to build paths, communicate with relays and encrypt packets for each relay and be sure that a majority of authorities concur on this information.

Selection from nodes eligible for a position in a path is random, weighted by the node's bandwidth, which is necessary to roughly balance loads across the network.

Additional weights are used to augment certain selections. The total bandwidth of guard nodes, exit nodes[6], nodes that are both guards and exits, and finally those nodes that are neither are separately calculated. A series of equations[41, Section 3.8.3] consisting of these calculated bandwidths and the as yet unknown weightings are used - balancing these equations to obtain values. These weightings are the probabilities used to select relays in path selection. Further instructions exist for various edge cases where relays or bandwidth of a particular type are in limited supply. One can imagine an adversary might dedicate some resources to attacking certain routers in the network if they believed the alternate rules outlined for one of these edge cases may be of benefit to them but that is beyond our scope. Users can alter various options to influence the paths that are created - the most common use being to ensure exit nodes are from a particular country. Generally any deviation from the default settings for Tor are heavily discouraged - even changes to the size of the browsing window - as it is likely to leak some amount of identifying information by reducing the number of nearly identical users they can hide amongst.

2.2.3 Notable Deviations from the Basic Operation

Bridges are a variation of a Tor node used for entry into the network intended for those where access to Tor is blocked or who otherwise want to hide the fact they are using Tor. IP addresses for bridges are not publicised but are obtained by emailing the Tor Project or using a site which applies rate limits. Bridges additionally attempt to obfuscate traffic so it cannot easily be determined to relate to Tor. Bridges are in limited supply and it is not terribly difficult for an adversary to gradually build up a list of active bridges.

The idea can be taken further with 'snowflake proxies' which are similar in core concept to bridges but are very simple to run by simply installing a browser extension. More people run these given the lower barrier of entry so there are more points to access the network and blocking is more difficult. Of course an adversary is equally able to also able to run more snowflake proxies than bridges with set resources[7].

Taking this to it's logical conclusion would be make every client also act as a router which would increase their number by perhaps two orders of magnitude at peak times. Such a design is not unheard of, the anonymising network I2P does exactly this[42]. It is hard to say what impact this would have on bandwidth and latency but most importantly, there could no longer

[6] Relay operators must opt-in to being available as the final node in a chain due to the risk of malicious activity then being seen to originate from an IP address they control.

[7] The exception being IP addresses at least when an IPv4 address is required.

be any hope of hiding the fact you were using Tor. Perhaps it could be opt-in only but without good adoption it wouldn't be more effective than the existing snowflake proxies. It may be that an ideal censorship resistant network and an ideal anonymising network cannot be one and the same - Tor makes a very good effort though.

3 Motivation and Goals

3.1 Motivation

In the preceding Section 2 we have discussed the societal role Tor plays and the technical reasons that necessitate its existence and argued - compellingly, we feel - for Tor's importance. Here we shall specifically discuss the rationale for a project considering an entry-exit linking attack and the robustness of Tor's path selection algorithm as the principal defence against it. Finally, we shall outline the motivations for using game theory to tackle this problem.

From the breadth of potential avenues for attacks on path selection we select entry and exit linking partly due to the relatively high availability of existing research. This selection provides wider applicability than focusing on a particular correlation attack whilst still being a well understood area that could expose the entire userbase if performed at scale.

Tor development has converged upon guard nodes as a mostly comprehensive solution to entry-exit linking and indeed similar attacks involving control of a portion of the network like attempting to control all three nodes in a path. As described in detail in the Guard Specification[43], when a Tor client is initially launched it will select a number of nodes from a subset of the network that have the 'guard' flag assigned. These selected guard nodes will be those that are exclusively used as the first node in any path that is established by the client and are intended to be used for an extended period of time. When users may be establishing dozens of paths a day, this technique can significantly reduce the probability of any particular user having a path containing compromised entry and exit nodes by reducing the number of different nodes they actually encounter. The guard selection algorithm minimises the number of users that any particular guard node has access to.

We consider this to be a well-intended and promising, yet - as we shall argue - flawed strategy. The principal issue with guard nodes is also the reason they are effective. Firstly, however strong the protection guard nodes afford to a majority of users there will always be another group that receive one or more compromised nodes as their long-lived guard and experience weakened security[8]. Secondly there are guard discovery attacks that seek to take advantage of a target's guard nodes should the adversary be unfortunate enough not to have one of their compromised nodes selected initially. By controlling the second and third nodes in a path an adversary can - without the use of more advanced linking techniques - discover the guard that certain traffic originated from. The adversary may tentatively assume that this particular guard node will continue to be used by their target for some time so may invest resources to hijack this node with malware, physically infiltrate the server location or conduct a denial of service attack such that the user must select another guard which has the potential to be one controlled by the adversary.

We have chosen to use game theory for our analysis as a tool to rationalise decisions and

[8] A compelling counterpoint to this argument may be that even if protection is only enhanced for a subset of users it disincentivises the entire attack vector thereby reducing the resources put into compromising nodes and further increasing the safety of the network. To the contrary we believe it equally likely to foster innovative variations of the existing attack.

provide insights into the interactions that may occur between those designing a path selection algorithm and those trying to circumvent it for their own gain - the adversary.

Thus far there has been limited investigation of this area with game theory, though equally there does exist some evidence of it being used effectively[44]. Given the high stakes involved for the thousands whose safety depends upon the defence of the Tor network we believe that it is vital that research in this area have some systematic means for reaching conclusions. We suggest that proposals for changes to Tor's path selection backed only by conjecture are therefore of fundamentally dubious value.

3.2 Overview of Goals

4 Related Work

In this section, we shall provide an overview of existing research and proposals that were influential to this project or important for general understanding of the topic. Firstly we will go into detail on several papers of preeminent relevance before reviewing works in several general research areas.

Hiding Routing Information[4] This is the initial description of the notion of Onion Routing from the Naval Research Laboratory which now has little resemblance to two the current network but retains the core idea of the routing architecture. The system here is envisioned operating at the application layer of the protocol stack with separate proxies for HTTP, SMTP and other protocols - this being opposed to operating with the TCP protocol at the transport layer as was later adopted. The relevance of this protocol choice is discussed in the next section.

There is a discussion of several vulnerabilities and the risk of compromised nodes - specifically pairs - is recognised and an explanation of measures to complicate correlating packets is mentioned - namely message expiration times and strict time synchronisation across the network intended to thwart replay attacks. Paths of arbitrary length are permitted which would increase latency for the negligible benefit of intermediary nodes being unaware of their position in any particular chain though inherently the entry and exit nodes must know their role so entry-exit attacks would not become more difficult.

The authors do, however, cite a 1981 paper by Chaum[45] as having the first inception of the general concept and one may argue that it is more accurate to call this the initial description of Onion Routing despite the name not yet being coined. With its presentation of a system in the context of email with a series of 'mixes' each with a public key readily available and willing packages with layers of encryption and then pass them on with the outermost layer removed aligns closely with the core ideas of the later work but lacks the detail and specifics to be readily considered a proposal for a routing system - a proof of concept implementation could certainly not be constructed with these ideas alone.

Tor: The Second-Generation Onion Router[46] The presentation of the Tor network as it is known today comes in a 2004 paper co-written by a member of the Naval Research Laboratory and two members of the, now defunct, Free Haven Project. Packet integrity checking is introduced preventing a branch of active correlation attacks where identifying data is inserted into a packet as it is processed by a compromised node. It should be clearly pointed out that defending against end-to-end attacks is explicitly stated to be beyond the threat model Tor was

designed for - which is a realistic choice for a network striving for low latency and mass adoption. Any attempts to provide these protections is therefore trying to add capabilities to a network that was never originally intended to support them and a vastly different core design could have provided greater anonymity. Nonetheless Tor is by far the most popular tool of its type and thus the focus of such a significant amount of research[47].

A crucial statement is "if an adversary controls $m > 1$ of N nodes, he can correlate at most $(\frac{m}{N})^2$ of the traffic"[46] which is ultimately core to the intentions of this project and indeed most work to improve the anonymity of Tor's path selection by finding a way to reduce $(\frac{m}{N})^2$ or make achieving that upper bound as resource intensive as possible.

A discussion of the advantages and disadvantages of moving from the application layer to the transport layer is also provided as well as another option which would have been to operate with the IP protocol at the network layer. Generally using IP would give a highly capable system that could be used natively by any transport level protocols though with poor anonymity characteristics whilst using HTTP would allow for several techniques to enhance anonymity but would limit the applications that could be interacted with and requiring various individual proxies to be developed as with the 1996 design. TCP provides a compromise in both these regards - settling for the middle ground where there are a spectrum of possibilities with conflicting priorities is indeed a doctrine that can be seen adopted throughout the design of Tor. For the most part it's a point consigned to history for both Tor and any comparable networks - working with IP is too difficult to implement as operating system kernel modifications would be required and HTTP is too constricted for a general purpose network - but it may be worth noting that entry-exit correlation would, broadly speaking, be easier if they had instead switched to IP and harder if they had continued using HTTP.

gPath: A Game-Theoretic Path Selection Algorithm to Protect Tor's Anonymity[44] Nodes are modelled only with a bandwidth and willingness to serve as exit nodes whilst a simplification of Tor's strategy sees them selected based only on weighted probabilities linked to the bandwidth they provide. Whilst broadly correct, there is significant additional nuance in how Tor goes about determining these probabilities though whether such nuanced details are likely to have practical impact in the strategies of either player remains to be seen.

The authors conceive two conflicting qualities of a path selection algorithm of privacy and utility. If nodes were selected in a completely random fashion there would be no way for the adversary to directly influence their probability of success other than simply acquiring more nodes[9] but the utility would be very poor with a single underperforming node in a path causing a bottleneck. These two qualities form the utility function for the algorithm designer.

Through a series of games, the authors settle upon three key pathing principles: stratified path selection, bandwidth order selection and adaptive exit selection. A combination of these principles leads to the proposal of the gPath algorithm that is suggested as an alternative to an existing algorithm used by Tor. In each subsequent game an additional principle is introduced by the defender and the adversary's best response is considered - though finding a Nash equilibrium is distinctly not within the goals of the paper.

[9]Whilst theoretically this sounds like an innocent solution for maximal privacy, but by giving no consideration to the capabilities of routers the adversary is limited only by the availability of IP addresses. Upon a single physical server they might run dozens of instances of Tor with permitting networking.

Finally a statistical analysis is performed to judge the developed pathing principles with an implementation of the algorithms and a snapshot of data from the Tor network.

An Overview of Texts on Game Theory In the fairly limited area of research concerning both the Tor network and game theory, there is a 2011 paper[48] proposing an attack correlating the requests and responses between particular users and a server using active fingerprinting to insert image tags and cookies with HTTP requests. Once that linking was discovered the adversary might be able find ways to exploit that information and ultimately deanonymise the user. The attack could be mounted with the control of only a single Tor exit node but the use of HTTPS renders such basic request manipulations impossible and with its prevalence increasing the applicability of this attack has only diminished. The game theoretic analysis is conducted using Gambit[49] - a set of tools for constructing and analysing models - with data obtained from a proof of concept implementation of their attack. This would be an interesting area for further research with large datasets.

Concerning use of game theory applications in routing, there is an analysis of server placement in the censorship circumvention technique Decoy Routing[50] The game consists of a network designer recruiting internet autonomous systems with their limited resources to make it difficult for a nation-state to route their citizen's traffic without it passing through at least one of these decoy routers. This has certain parallels to our path selection problem albeit in reverse with the Tor designers in the position of the censoring nation and the adversary trying to find a 'safe' path knowing the identity of all 'compromised' nodes. Particularly since it isn't a consideration of gPath, finding a Nash equilibrium in a practical setting relatively similar to ours would prove helpful were we to make such an attempt - there is discussion of their methods and an algorithm they use in this undertaking. Also of interest as a technique is their implementation of simulation for the routing to use with the game theoretic algorithms they describe.

Additionally, there is research of path selection between mobile devices with the intent to detect malware[51] and a more generalised case considering a malicious intruder into a network[52]. Together with gPath we feel this demonstrates game theory as a feasible and powerful tool for exploring challenges in path selection and routing across a variety of circumstances.

An Overview of Texts on Tor There is research considering the issue of weighting in the path selection algorithm considering only total bandwidth and not the load a particular node is currently receiving[53]. Considering various metrics of load is shown to be capable of providing significant increases in network throughput and potentially anonymity enhancements. Equally, there could also be unintended consequences, some of the described metrics could be easy for an adversary to manipulate on their own server, such as reported CPU load, and such a system could enhance the utility of denial of service attacks to drive more traffic to compromised nodes.

Of particular severity is a path bias attack to enhance a entry-exit correlation attack where a malicious guard node would intentionally disrupt connections if they are being made to an exit node that wasn't also controlled by the attacker so new circuits (still using the same guard) would need to be constructed[54]. In the worst case the adversary could increase the proportion of circuits compromised from $\left(\frac{m}{N}\right)^2$ as described above to merely $\frac{m}{N}$. The Tor path specification[55] outlines measures for such an attack to be detected though these have yet to be implemented for technical reasons. We will most likely ignore this from our analysis under the assumption an upcoming release will rectify the vulnerability however it does raise the particularly interesting question as to whether a path selection algorithm can be devised that does not require guards

but remains competitive in preventing compromise.

A breadth of research exists showing the feasibility of both passive and active correlation attacks to perform entry-exit linking[56, 57, 58, 59, 60, 61] and thus we feel it is certainly an important area for the analysis of existing defences and discussion of enhancing those defences. Indeed, as mentioned in the introduction there has even been discovery of what appeared to be an extremely similar attack operating 'in the wild' on the Tor network[62]. It is not important for us to understand the specifics of any particular technique used in these attacks, merely to be aware that it is indeed a realistic threat.

5 Game Theoretic Analysis

5.1 Introduction

Developing upon the work in gPath[44] we have constructed a more complex model of the Tor network including more recent developments - namely Guard nodes. Our design has also been heavily influenced by the needs of our co-objective of creating an analysis program. Introductions to algorithmic game theory and game theory more generally are provided by respective texts[63, 64].

For our analysis we have constructed a non-cooperative game where parties (algorithm designer and adversary) are solely self-serving and it will be a game of perfect information. The fact that the adversary will know the strategy of the defender is clear since any detail will be trivial to discover from inspecting Tor's open source code. The inverse is more difficult to justify as any particular attack would need to maintain secrecy lest a concerted effort be undertaken to remove compromised nodes from the network. Ultimately, however, our interest is in resilience to the best known attack methods so this is of little relevance. Relatedly, Tor must of course be the first player to make any move as there must be an algorithm proposed before any attempt can be made to circumvent its intended protections though any potential impact would be negligible over repeated rounds.

Whilst it would have been possible to alternatively model this as an extensive form game with the designer garnering only limited information on the strategy of the adversary before they must make the next round of modifications to their algorithm but this is an area for further research.

Of course, there is a clear incentive to look for a Nash Equilibrium - where neither player is motivated to deviate from their strategy - to discover a best response for the algorithm designer. This isn't a goal of gPath and as they discuss one may encounter difficulties with the interactions between the player's relatively detailed strategies[44, 65] thus it is also something we have not attempted to explore.

5.2 Modelling Our System and Developing Framework

5.2.1 Tor Network

Firstly we must have a construction of the Tor network to work upon that strikes a balance between having low enough complexity that it is feasible to interact with and theorise about whilst having enough depth that our results remain applicable to reality.

Consider that we have n Tor nodes $\mathcal{R} = \{r_0, ..., r_{n-1}\}$. A path may be denoted with a 3-tuple

of routers $p = (r_0, r_1, r_2)$. The order in which routers are selected for the different positions in a path is variable as well as the pool of nodes available for each selection. In the default case, routers are selected sequentially and the pools will be all nodes baring only those that have already been selected for the given path as seen here:

$$
\begin{aligned}
r_0 &\in \mathcal{R} \\
r_1 &\in \mathcal{R} \setminus \{r_0\} \\
r_2 &\in \mathcal{R} \setminus \{r_0, r_1\}
\end{aligned}
$$

Discretion may be expressed as to the inclusion of a given node in a particular pool based on the attributes that are applicable to that node. Many of these are as defined in Tor's Directory Specification[41] though we will discuss only those that may be influential to our analysis. A note on terminology: a boolean attribute shall be referred to as a 'flag'.

We shall discuss some techniques an adversary may use to influence attributes they do not have direct control over and ultimately form an abstraction of only those attributes we believe have pertinence to the model we are constructing.

Fast A node is considered *Fast* if it has either a bandwidth greater than 0.78 Mbps or its bandwidth places it within the top 87.5% of all nodes. This is a fairly outdated flag and a majority of nodes fulfil this requirement - for perspective the average upload speed in the UK is 10.7 Mbps[66] so the typical household could theoretically run a dozen nodes considered *Fast* (In the dataset used for reaching the conclusions in this paper, 93.4% of nodes have the *Fast* flag). Nonetheless, if the adversary did have cause to manipulate this metric, they might create a large number of nodes with negligible bandwidth such that it becomes easier to place within top 87.5%. More onerous bandwidth requirements are available in our model by setting a minimum so there would be little benefit to including *Fast*.

Exit This flag controls whether a node is eligible to appear as the final node in a circuit. It is at the discretion of the node's operator.

MiddleOnly, BadExit and Sybil *MiddleOnly* and *BadExit* are somewhat nebulous values that authorities have the discretion to vote on without any clear, public guidelines on precisely when this should occur. As the names suggest the first limits nodes to only position 2 in a circuit and second limits nodes to positions 1 or 2. These flags are available for nodes that have somehow been identified to been acting inappropriately through either inadvertent error by the operator or malice. Any benefits arising from not simply removing these nodes from the network entirely are unclear and given the imprecise and subjective determinations that may go into deciding a node is acting inappropriately, we fail to see an effective way this might be incorporated into the model. An argument could be made for handling the risk of detection as a cost to the adversary that scales with the proportion of nodes they control or the proportion of users they wish to target but that remains very difficult to quantify. *Sybil* is a flag assigned to all bar one of a set of nodes originating from a single IP address which is similar to those nodes simply being ignored so in our model we shall just consider that each node must have a unique IP address.

Stable Determined from MTBF (Mean Time Between Failures) which is weighted to give greater influence to recent stability of the node. This differs from considerations of uptime alone

in that it would provide an impediment to the adversary using techniques that require the intentional dropping of certain connections. For example if the adversary detects that exactly one of its nodes is in a particular circuit, they may wish to sabotage the connection so the adversary is forced to select another circuit - providing an additional opportunity for a compromised circuit to be formed.

Guard Discarding those parameters mentioned above, we are left with the following requirements for a node to be considered a guard: *Stable*, *Familiar* - either older than at least 12.5% of nodes or older than several weeks, Weighted Fractional Uptime (WFU) if greater than the median of all familiar nodes, and it has a bandwidth of at least 2 MB or a bandwidth greater than 25% of routers - we can refer to this as VeryFast VF() Regarding WFU we are unable to see any scenario where intentionally running a node only a limited amount of the time would present an advantage so we abstract having reasonable uptime as part of the cost to the adversary of running a node.

Surmising Our Model Following this rationale, we are left with three parameters for a node. $E(r)$ if the node r has the parameter *Exit*; $S(r)$ if r is *Stable*; and $G(r)$ if r is a *Guard* such that $\forall r \in \mathcal{R}(E(r) \wedge S(r) \wedge F(r) \wedge VF(r) \Leftrightarrow G(r))$.

Consider a set of trilean (having three possibilities) values: $\mathcal{T} = \{E, S, G\}$. Let $\bot$ indicate selection of only nodes that do not have the property, $\top$ be selection of only nodes that do have the property and $\perp\!\!\!\perp$ be selection of all nodes regardless of whether they have the property (indifference). We have the function $R()$ which takes this set of trileans and a set of nodes and outputs a set containing only those nodes that adhere to the restriction set[10]. We can further add to our restriction function a minimum and maximum bandwidth: $R(\mathcal{T}, b_{min}, b_{max}, \mathcal{R})$ We call the output of a restriction function a Restricted Node Set. We use the notation $E(\top)$ to say 'must be an exit', $E(\bot)$ and $E(\perp\!\!\!\perp)$ for selected independent of whether it is an exit - and likewise for the other flags.

There are two metrics that may be used to analyse bandwidth. The maximal bandwidth of a path is equal to the minimum of the three bandwidths from the involved nodes. The maximal bandwidth of a client is the is the summation of the maximal bandwidth of all the paths that they have formed. The first measure will incentivise maximising the bandwidth of each path, the second will incentivise having clients form many paths which must be balanced with a growing risk of compromise. For this project we will only model paths and their bandwidth. We normalise the bandwidth of nodes in to the range $[0, 1]$ where 1 is the highest observed bandwidth in this set. The total bandwidth of all paths modelled is $\mathcal{B}$.

5.2.2 Adversary

We shall consider $\mathcal{A} \subseteq \mathcal{R}$ to be the set of routers under control of the adversary through any means; be it creating their own servers, installing malware on those that have been created innocently or otherwise.

$\mathcal{P}$ shall be the set of all paths formed in the network and $\mathcal{P}_c$ will be the set of paths formed by a particular client. A path is considered compromised if both the first and third node are controlled by the adversary and a client is considered compromised if any of the paths they form

[10]This is simply saying that we may limit ourselves to only nodes that are both Guards and Exits for example.

are compromised. We can formalise this with $Comp(p)$ to denote that the path $p = (r_0, r_1, r_2)$ is compromised: $(r_0 \in \mathcal{A} \wedge r_2 \in \mathcal{A} \Leftrightarrow Comp(p))$ and $Comp(c)$ where $\exists p \in \mathcal{P}_c(Comp(p))$.

The strategy for the Adversary consists of rules for building the set of nodes that they will compromise. The adversary will have one or many Restricted Node Sets - each defined by a Restriction Function - and proportions of how many of their nodes should be sourced from each set[11]. Together with a definition of the resources available to the adversary to compromise nodes, this defines the set of compromised nodes.

The adversary has two complementary but differing objectives: to increase the proportion of $\mathcal{P}$ that is compromised and increase the probability that any given $\mathcal{C}_{path}$ contains at least one compromised path. In the default environment, progress in regard to either goal necessarily equates to progress with both but this relationship may be broken by certain defender strategies - namely using guards.

Consider the utility function that the adversary wishes to maximise:

$$U_A = \left(\frac{|Comp(\mathcal{P})|}{|\mathcal{P}|}\right)(1 - \mathbf{P}) + \left(\frac{|Comp(\mathcal{C})|}{|\mathcal{C}|}\right)(\mathbf{P})$$

Where $\mathbf{P} \in [0, 1]$ is the adversaries' preference for the compromise of clients over the compromise of nodes.

5.2.3 Tor Network Designers (Defender)

The strategy for the defender consists of defining a path selection algorithm for Tor. They must define the order in which each node in a path will be selected and the Restricted Node Set from which each node in the path shall be selected. Ordering shall be shown with a 3-Tuple of the node positions - $< 1, 2, 3 >$ would be selecting them in the obvious sequential order.

Crucially the Restriction Function may be varied on a per-node basis namely such that bandwidths of previously selected nodes in a particular path limit later node selections[12] - this is a key element of the gPath algorithm[44].

Generally the defender wishes to maximise bandwidth and minimise compromise - for both of which we have to separate measures - as previously described - of which none are objectively more important that the others.

Consider the utility function that the defender wishes to maximise (limited here to considering paths):

$$U_A = \left(1 - \left(\frac{|Comp(\mathcal{P})|}{|\mathcal{P}|}\right)\right)(1 - \mathbf{P}) + \left(\frac{\mathcal{B}}{|\mathcal{R}|}\right)(\mathbf{P})$$

[11] For example an adversary might wish for every node to be a Guard or they might wish for an even split between Guards and Exits with 50% of resources allotted to each.

[12] For example, it may be stated that the exit node must have the highest bandwidth of those in a particular path.

Where $\mathbf{P} \in [0, 1]$ is the defender's preference for the preventing compromise over preserving good network performance - high bandwidth. Multiple measures results in more complex utility functions however allow for a wide breadth of analysis within our framework. There is an accuracy trade-off in increasing ease of analysis by focusing on just the measure of compromise. In taking such a 'shortcut', caution of extreme solutions is imperative - a path selection algorithm that rejects all routers and creates no paths would be immune to compromise but would also completely fail to provide a network that fulfils any purpose. Generally we feel the cost of considering the full utility function is worthwhile.

5.3 Modelling Defender Strategies

5.3.1 Defender's Strategy to Replicate gPath[44]

We begin by creating in our framework, the modelling of Tor that was used in creating gPath:

- Order $3, 1, 2$
- Restriction Function for node 3: $\mathcal{T} = \{E(\top), S(\Subset), G(\Subset)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 1: $\mathcal{T} = \{E(\Subset), S(\Subset), G(\Subset)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 2: $\mathcal{T} = \{E(\Subset), S(\Subset), G(\Subset)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.

This is a very simple strategy where the only rule used other than ordering is that the final node must be an exit which is a necessity.

The gPath algorithm within our framework may be expressed with the following strategy which we believe aligns almost exactly with the original definition[13]:

- Order: $< 2, 1, 3 >$
- Restriction Function for node 2: $\mathcal{T} = \{E(\bot), S(\Subset), G(\Subset)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 1: $\mathcal{T} = \{E(\bot), S(\Subset), G(\Subset)\}$ and $b_{min} = \varnothing, b_{max} = b_2 + 1$[14]. Select randomly by bandwidth weighting.
- Restriction Function for node 3: $\mathcal{T} = \{E(\top), S(\Subset), G(\Subset)\}$ and $b_{min} = \varnothing, b_{max} = b_1 + 1$. Select randomly by bandwidth weighting.

Expressed in plain English: Firstly select the second node, which must not be an Exit. Then select the first node, which must not be an Exit and must have bandwidth less than or equal to the first. Finally select the third node which must be an Exit and must have less bandwidth than the second node.

5.3.2 Defender's Strategy to Replicate Modern Tor

The most impactful difference to the gPath strategy is the use of the guard system. We can model this only so far as entry nodes needing to adhere to the requirements for guards as we are not modelling clients and their guard selection process. We argue that for analysis at the path

[13] gPath selects nodes for positions 1 and 2 together and then assigns them to their specific position by comparing their bandwidth - this may provide marginally more effective use of the network's bandwidth.

[14] The addition of one arises from gPath specifying bandwidth may be less than or equal which is something we cannot natively represent in our model.

level this will provide a good approximation with a sufficient number of clients each creating a many of paths. As we have discussed previously, use of Guards excels as reducing risk to any particular user but not to any particular path nor necessary the network as a whole.

Within the limits of our framework we approximate the modern Tor algorithm as the following strategy:

- Order $3, 1, 2$
- Restriction Function for node 3: $\mathcal{T} = \{E(\top), S(\perp\!\!\!\perp), G(\perp\!\!\!\perp)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 1: $\mathcal{T} = \{E(\perp), S(\perp\!\!\!\perp), G(\top)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 2: $\mathcal{T} = \{E(\perp), S(\perp\!\!\!\perp), G(\perp)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.

Expressed in plain English: Firstly select the third node, which must be an exit. Then select the first node, which must be a guard and may not be an exit. Finally select the second node which may neither be an exit nor a guard.

Even beyond the point about guards, this is less accurate than our modelling of gPath - though as an algorithm itself developed for a simplified model that is to be expected.

Some of the limiting rules here - namely the exit restriction for the first node; and the exit and guard restriction on the second node - are not explicitly stated in the specification of Tor but controlled by the consensus weights described in Section 2.2.2(setting a weight to 0 is equivalent to eliminating it from selection). Thus it is more transient and could be changed frequently though this is not observed to be the case and as a tool that is indeed available to Tor's network designers, we choose to include it. In particular, the weight for a node in the 2nd position that is an exit is 0 but weights may also apply to multiple parameters and the weight for a node in the second position that is both an exit and a guard is 0. Therefore it is perfectly reasonable to have a guard node in the 2nd position as long as it is not also an exit - this nuance is not something that can be replicated with our model and we must always prohibit guards or always permit them for a given position. In this case we have chosen to go with the more restrictive option and not allow any guards in position 2. The result is adherence to the first principle of gPath - that the 1st and 3rd nodes should be drawn from separate pools[44] - and in fact exceeds it by having 2nd nodes drawn by yet another mutually exclusive pool. Whilst this would make it more difficult for an adversary to control all three nodes in a path, it is also a less effective use of the network's overall bandwidth. The degree to which designers should be focused on tackling entry-exit linking (requiring less compromised resources but more technical complexity) vs complete path compromise (requiring more compromised resources but less technical complexity) is an area for further research.

Additionally some weights are calculated by formula and reduce the prevalence of nodes with certain parameters being used in certain positions - for example the likelihood that a guard be selected as the 3rd node decreases as the number of guards active in the network decreases and they become a more valuable commodity to be reserved to act more exclusively as guards[41]. Again we cannot replicate this in our framework.

Nonetheless, we feel this model is certainly accurate enough for useful insights to be formed.

6 Implementation

We approximately implemented the rules of our game theoretic framework into a command line application that obtains the current state of the Tor network from directory servers and allows user-configured path selection rules to be tested against adversary strategies in a realistic environment of genuine Tor nodes.

6.1 Analysis Framework

Broadly, the framework implemented follows that described in the preceding section to make insights from one as applicable to the other as possible. Some additions and deviations were necessary, which will be discussed here.

In addition to the concept of paths and clients being Compromised we introduce the idea of 'failure' where the Restriction Function is so strict as to leave no available nodes for selection. This is particularly prevalent where large sets of nodes are being formed (consider attempting to select 2,000 Exit nodes to compromise where only 1,500 Exit nodes exist) and where nodes being selected for a path are dependent on nodes already selected (consider the node with the highest of all bandwidths being selected first but the rules dictate that subsequent nodes must have higher bandwidth). Equally it is possible that a poor Restriction Function has been constructed that no nodes adhere to. Compromised nodes, Guard nodes, and paths may all exhibit failure. An adversary is failed if any attempts to select a compromised node fail. A client is failed if selection of any guards or paths fail.

For the dataset to best reflect the Tor nodes actually in circulation and being used as paths, some must be removed from the full list obtained in the Tor consensus. Namely, after obtaining a consensus, nodes with the flags Authority, MiddleOnly and BadExit are removed as well as any nodes that don't have the flags Valid and Running.

6.2 Program Design

Our program was developed in Java, namely OpenJDK 20 principally because it is the language we were most familiar with and as there was no compelling advantages to using an alternative for this particular project.

We used SQLite as the database for storing nodes and their attributes. With only six to seven thousand nodes and limited complex query needs, this seemed more desirable that a more full-featured database engine. This was vastly more performant and simple to query than directly accessing the consensus document or an object oriented solution. Two external dependencies were used: The Java Database Connectivity driver for SQLite and JCommander which was used to parse command line arguments and some parts of configuration files.

Using the command line, a user requests that the consensus be downloaded from a particular Tor Directory Server and provides a name for this to be saved under. We parse the consensus document and construct a database table of nodes, any nodes that are unsuitable for use are removed. The user can then use the command line to perform analysis with any previously obtained consensus tested against a file they provide defining the adversary's strategy and a file defining the defender's strategy as well as simulation parameters. An adversary is created which selects and stores compromised nodes according to the provided rules and the quantity requested - via a percentage or absolute value for either the number of nodes or the total bandwidth of nodes.

The requested number of clients are created which will each select nodes for the requested number of paths and guards if the path selection rule set uses them. Each client is a thread so they can run concurrently which greatly benefits performance when simulating thousands of clients with multicore processors. A report is provided to the user detailing the effectiveness of the strategies with the proportions of clients and paths that were compromised and the compromised bandwidth.

7 Interactions

7.1 Analysing gPath

Firstly, in our testing we did not observe gPath performing better than a default defender strategy (the only rule being that the third node must be an exit) when both are tested against an 'ignorant' adversary (one that has no strategy and compromises nodes arbitrarily). In our tests we targeted 20% of network bandwidth for compromise and created 5,000 clients each creating 10 paths. We performed that test 10 times for the default and gPath resulting in an average of 277.3 and 271.8 compromised paths respectively. Using the Levene Test, we confirmed the data was homogeneous so suitable for the unpaired student t-test. We use a two tailed test and get a p value of 0.95 which far exceeds the typically accepted value of 0.05 for significance so suggests that using gPath doesn't result in a statistically significant difference. Intuitively the pool separation of gPath should be beneficial but we theorised that the bandwidth restrictions may be undesirably limiting available nodes. Testing a minimal version of gPath without the bandwidth restrictions with the same methodology as above resulted in an average 192.8 path compromises - comparing to standard gPath has a p value of 0.27 so it isn't statistically significant either.

Returning now to looking at gPath itself: there are two pools of nodes, those that are Exits and Not-Exits. Our Exits should have bandwidth less than or equal to our Not-Exits to use our resources as efficiently as possible (In every case where one of our Not-Exits is selected in position 1, every Exit we have will be in the pool for selection) so we will set the maximum bandwidth for an Exit equal to minimum bandwidth for a Non-Exit - we have a single value Bandwidth.

We must consider how resources should be shared between the two pools and selection of Bandwidth. Initially we consider splitting our resources such the proportion of the total number of Not-Exits we control is equal to the proportion of the total number of Exits with bandwidth less than or equal to Bandwidth - there is no point being competitive with Exits that have greater bandwidth because that can only matter when one of our Not-Exits has definitely not been selected for position 1 so we do not care.

Setting Bandwidth extremely high increases chances in bandwidth weighted selection but we also need to have a bandwidth that is less than or equal to that selected for position 2. We consider a selection of either the mean or the median of bandwidths. Unfortunately here we encounter a limitation of the design of our program using adversary selection from existing Tor nodes - there are insufficient nodes with the applicable attributes to effectively test adversary strategies of any interesting complexity. Of course rates of path compromise do not scale linearly with rates of node compromise so results with lower levels of compromise cannot simply be extrapolated.

7.2 Analysing Modern Tor (The Guard System)

Following the design goals of guards, we will be analysing the strategy with the criterion of number of clients compromised rather than the number of paths - a client with one compromised path necessarily has a compromised guard which will be reused so has a likelihood of forming multiple compromised paths. We will be testing with clients that select 3 guards and create 30 paths to ensure we can observe this effect fully.

The most obvious attack vector for a strategy using guards is for the adversary to focus on compromise of guards. We will test a basic guard implementation where the only rules are that the first node must be a guard and the third node must be an exit. As before the tests will consist of 5,000 clients and be performed 10 times. Incidentally, the reason for not just doing a single test with 50,000 clients (aside from the issue with memory utilisation that we discuss in the product evaluation) is to account for the compromised nodes being randomly selected. We test three adversary strategies: the default strategy mentioned above, selecting only guards for compromise and finally selection only nodes that are both guards and exits for compromise. Respectively these result in average client compromise numbers of 230.5, 240.7 and 296.1. The difference between default and only guards is barely perceptible with a p value of 0.995. The difference between default and only nodes that are both exits and guards is 0.026 which is less than 0.05 so this result is considered to be statistically significant - we can be fairly confident in saying that this adversary strategy is indeed effective against this defender strategy.

If the section on modelling modern Tor in our framework is revisited, it can be noted that there is resistance to the latter two adversary strategies described above with the restriction of exit nodes not being permitted in the first position. We can still tighten restrictions even further by dictating that guards may not appear in the third position, this ensures that entirely different pools are required like gPath. Additionally we rearrange the selection order so the middle node is selected first and remove all pool restrictions from the middle node. Our rationale is that compromised nodes selected here are now wasted as they cannot appear again in that path - this would be effective for the adversary controlling a very small number of very high bandwidth nodes that have a significantly weighted likelihood of being selected. Whilst a minor effect we think it may be perceptible at a large enough scale[15].

Consider our adjustments to modern Tor set out in our modelling notation:

- Order $2, 3, 1$
- Restriction Function for node 2: $\mathcal{T} = \{E(\perp\!\!\!\perp), S(\perp\!\!\!\perp), G(\perp\!\!\!\perp)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 3: $\mathcal{T} = \{E(\top), S(\perp\!\!\!\perp), G(\perp)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.
- Restriction Function for node 1: $\mathcal{T} = \{E(\perp), S(\perp\!\!\!\perp), G(\top)\}$ and $b_{min} = \varnothing, b_{max} = \varnothing$. Select randomly by bandwidth weighting.

Testing modern Tor against my adjusted version using the same methodology as previously against the default adversary results in average client compromises of 225.1 and 199.4 respectively. Compelling enough for further investigation in this area to be directed but inconclusive - comparing them gives a p value of 0.12 so it is not statistically significant.

[15] Clearly this is not giving consideration to full path compromise of all nodes - this would slightly weaken defences against that form of attack.

To summarise, an adversary that focuses on nodes that are guards and exits is going to be relatively effective in attacking a guard based defence if not mitigated by extra rules. Modern Tor does have such additional rules but still does not achieve complete subset separation for selection of first and third nodes. Preliminary investigation suggests that implementing that complete separation could be beneficial but further research is required.

8 Evaluation

8.1 Implementation

Our program achieved the core goals but there are some areas we feel should have been more aspirationally targeted and we did not complete the aspirational goal of statistical analysis capabilities.

We were originally highly content with our command line interface and we still don't feel it is a serious hindrance to usability but with additions of more features it will become increasingly desirable to streamline the input of complex strategies and unlock the ability to produce graphs as part of analysis with a GUI.

It is clear that the program would be a far more helpful research tool if many tests could be performed at once and even with a variable moving across a range. Additionally statistical information like standard deviation for results and ideally performing stats tests to compare different strategies would relieve the researcher of performing manual calculations like those necessary for the results in the preceding section. To fully realize both of these points would require a redesign to have more measurements on the bandwidth if we wanted to target that for deeper analysis and performance concerns would also need to be addressed.

As we discussed in the section on program design we were mindful of how our choices would impact performance however we failed to fully realise this as a limitation of the program we were developing. Clients store more data than is strictly necessary and that results in memory usage exceeding 10 GB in some typical tests and even program crashes due to lack of memory when running some larger tests on a machine with 32 GB memory. This could have been avoided by including more of the analysis in the client to be performed in 'real-time' rather than after all clients had finished path selection. Since the clients are multithreaded, this almost certainly would have also resulted in faster runtimes. Whilst it may be tempting to blame this oversight on using an incremental software development model[67] where major changes became more costly to carry out as time went on, it is more likely to simply be our inexperience in large, independent software development projects at the outset. A database solution that is heavily focused on the efficiency of many concurrent read operations may also be desirable. Whilst SQLite is undoubtedly very capable, consider that the very small amount of data presented in the analysis of modern Tor required the execution of at least 22.5 million SELECT statements which would increase significantly with more complex analysis - so even a small improvement could be very beneficial.

Finally we have a limitation not of program design necessarily but more our overall methodology for this research using actual data. Being limited to actual, real Tor nodes greatly reduces the adversary strategies that can be tested without running out of suitable nodes which is particularly unfortunate as this was a key area we were intending to research.

8.2 Processes and Project Overall

Competence of project management didn't prove to be a hindrance to completion of the project but could certainly have seen improvement. Actual completion times for milestones on our schedule vary wildly from what we targeted - sometimes overestimating the work required for a particular component and sometimes underestimating. In particular we feel that a software development model with a greater emphasis on planning in advance could have alleviated some of the issues we encountered.

We were not fully satisfied with the depth of our game theoretic analysis. The level of interweaving between game theory and practical analysis that we wished to carry out was severely restricted by the limitations of the program we developed. We were, however, very happy with our modelling and analysis of game theory in gPath[44].

Whilst it is disappointing that only one of our tests showed statistically significant correlation we feel that negative results and positive results are equally important and this provides a very solid foundation for continuing research into path selection in the Tor network.

Overall whilst there are numerous changes that could have been made we equally don't regret any choices in the project as we feel the lessons learned from the experience as a whole were invaluable. We hope to apply these lessons to great effect in our Masters project in the coming academic year.

Thank you for reading and I would like to thank my project supervisor Dr Arno Pauly for their invaluable guidance, particularly in the areas of game theory and academic writing.

A Addendum, 2026

The preceding dissertation is presented as it was originally submitted except for orthographical corrections.

References

[1] *The Tor Project — Privacy & Freedom Online*. URL: https://torproject.org (visited on 07/10/2022).

[2] *Onion Routing: History*. 2005. URL: https://www.onion-router.net/History.html (visited on 07/10/2022).

[3] Dune Lawrence. "The Inside Story of Tor, the Best Internet Anonymity Tool the Government Ever Built". In: *Bloomberg* (24th Jan. 2014). URL: https://www.bloomberg.com/news/articles/2014-01-23/tor-anonymity-software-vs-dot-the-national-security-agency (visited on 08/10/2022).

[4] David M. Goldschlag, Michael G. Reed and Paul F. Syverson. "Hiding Routing Information". In: *Information Hiding*. Ed. by Ross Anderson. Red. by Gerhard Goos, Juris Hartmanis and Jan van Leeuwen. Vol. 1174. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 137–150. ISBN: 978-3-540-61996-3 978-3-540-49589-5. DOI: 10.1007/3-540-61996-8_37.

[5] Ian Goldberg and Adam Shostack. *Freedom Network 1.0 Architecture and Protocols*. URL: https://shostack.org/files/zks/zeroknowledgewhitepapers/arch-tech.pdf.

[6] Andrew Orlowski. *Freedom Network Source Code Now Available*. URL: https://www.theregister.com/2002/02/15/freedom_network_source_code_now/ (visited on 13/04/2023).

[7] *Matej Pfajfar - Chief Technology Officer (CTO) - Monzo Bank — LinkedIn*. URL: https://uk.linkedin.com/in/mpfajfar (visited on 12/04/2023).

[8] *The Tor Project — History*. URL: https://www.torproject.org/about/history/ (visited on 09/10/2022).

[9] *'Tor Stinks' Presentation – Read the Full Document*. The Guardian. 4th Oct. 2013. URL: http://www.theguardian.com/world/interactive/2013/oct/04/tor-stinks-nsa-presentation-document (visited on 05/10/2022).

[10] Pierluigi Paganini. *How Edward Snowden Protected Information ... and His Life*. Infosec Resources. 25th July 2013. URL: https://resources.infosecinstitute.com/topic/how-edward-snowden-protected-information-and-his-life/ (visited on 08/10/2022).

[11] *Transparency, Openness, and Our 2020-2021 Financials — Tor Project*. URL: https://blog.torproject.org/transparency-openness-and-our-2020-and-2021-financials/ (visited on 17/04/2023).

[12] Julian Borger. "CIA Controlled Global Encryption Company for Decades, Says Report". In: *The Guardian. US news* (11th Feb. 2020). ISSN: 0261-3077. URL: https://www.theguardian.com/us-news/2020/feb/11/crypto-ag-cia-bnd-germany-intelligence-report (visited on 10/10/2022).

[13] Scott Shane and Tom Bowman. "RIGGING THE GAME Spy sting: Few at the Swiss factory knew the mysterious visitors were pulling off a stunning intelligence coup – perhaps the most audacious in the National Security Agency's long war on foreign codes; NO SUCH AGENCY". In: *Baltimore Sun* (10th Dec. 1995). URL: https://www.baltimoresun.com/news/bs-xpm-1995-12-10-1995344001-story.html (visited on 10/10/2022).

[14] David Harborth. *HOW PRIVACY CONCERNS, TRUST AND RISK BELIEFS AND PRIVACY LITERACY INFLUENCE USERS' INTENTIONS TO USE PRIVACY-ENHANCING TECHNOLOGIES - THE CASE OF TOR*. URL: https://www.pape.science/files/publications/HP20sigmis.pdf.

[15] Janis Dalins, Campbell Wilson and Mark Carman. "Criminal Motivation on the Dark Web: A Categorisation Model for Law Enforcement". In: *Digital Investigation* 24 (1st Mar. 2018), pp. 62–71. ISSN: 1742-2876. DOI: 10.1016/j.diin.2017.12.003.

[16] Clement Guitton. "A Review of the Available Content on Tor Hidden Services: The Case against Further Development". In: *Computers in Human Behavior* 29.6 (1st Nov. 2013), pp. 2805–2815. ISSN: 0747-5632. DOI: 10.1016/j.chb.2013.07.031.

[17] Tom Fox-Brewster. "Silk Road 2.0 Targeted in 'Operation Onymous' Dark-Web Takedown". In: *The Guardian. Technology* (7th Nov. 2014). ISSN: 0261-3077. URL: https://www.theguardian.com/technology/2014/nov/07/silk-road-20-operation-onymous-dark-web-drugs-takedown (visited on 18/04/2023).

[18] *AlphaBay Takedown*. Federal Bureau of Investigation. URL: https://www.fbi.gov/news/stories/alphabay-takedown (visited on 18/04/2023).

[19] Thomas Brewster. *'The Underground Golden Age Is Over': Epic Dark Web Opioid Bust Sees 179 Arrests And $6.5 Million Seized*. Forbes. URL: https://www.forbes.com/sites/thomasbrewster/2020/09/22/epic-dark-web-bust-sees-179-arrests-and-65-million-seized/ (visited on 18/04/2023).

[20] Kevin Poulsen. "FBI Admits It Controlled Tor Servers Behind Mass Malware Attack". In: *Wired* (). ISSN: 1059-1028. URL: https://www.wired.com/2013/09/freedom-hosting-fbi/ (visited on 18/04/2023).

[21] Gemma Davies. "Shining a Light on Policing of the Dark Web: An Analysis of UK Investigatory Powers". In: *The Journal of Criminal Law* 84.5 (1st Oct. 2020), pp. 407–426. ISSN: 0022-0183. DOI: 10.1177/0022018320952557.

[22] *Thoughts and Concerns about Operation Onymous — Tor Project*. URL: https://blog.torproject.org/thoughts-and-concerns-about-operation-onymous/ (visited on 18/04/2023).

[23] Kashmir Hill. *How Did The FBI Break Tor?* Forbes. URL: https://www.forbes.com/sites/kashmirhill/2014/11/07/how-did-law-enforcement-break-tor/ (visited on 18/04/2023).

[24] *NOTICE OF PROPOSED RULEMAKING*. URL: https://transition.fcc.gov/Bureaus/Common_Carrier/Notices/1999/fcc99283.pdf (visited on 02/04/2023).

[25] Chris Velazco. "FCC Calls 25 Mbps 'Broadband' Speed. The Push Is on to up It to 100." In: *Washington Post* (20th July 2022). ISSN: 0190-8286. URL: https://www.washingtonpost.com/technology/2022/07/19/fcc-broadband-new-definition-100mbps/ (visited on 02/04/2023).

[26] Christopher Allen and Tim Dierks. *The TLS Protocol Version 1.0*. Request for Comments RFC 2246. Internet Engineering Task Force, Jan. 1999. 80 pp. DOI: 10.17487/RFC2246.

[27] *Users – Tor Metrics*. URL: https://metrics.torproject.org/userstats-relay-country.html (visited on 17/04/2023).

[28] "A Guide to Using SecureDrop". In: *BBC News. UK* (19th May 2022). URL: https://www.bbc.com/news/uk-60972903 (visited on 10/10/2022).

[29] *The Guardian SecureDrop*. 2019. URL: https://www.theguardian.com/securedrop (visited on 10/10/2022).

[30] "Tips". In: *The New York Times. U.S.* (14th Dec. 2016). ISSN: 0362-4331. URL: https://www.nytimes.com/tips#securedrop (visited on 10/10/2022).

[31] *Secure Drop*. URL: https://securedrop.org/ (visited on 12/10/2022).

[32] *About Twitter.Com Supported Browsers*. URL: https://help.twitter.com/en/using-twitter/twitter-supported-browsers (visited on 11/10/2022).

[33] *Facebook over Tor*. 24th May 2021. URL: https://www.facebook.com/facebookcorewwwi/posts/pfbid0evxYMitgBDfP3qSvRoC7Q2tjAFhPVPWhtq8FZUwbLRGX1PkLthAa5KHSG2khtjwcl (visited on 11/10/2022).

[34] *Central Intelligence Agency (CIA)*. 2nd May 2022. URL: https://www.facebook.com/Central.Intelligence.Agency/posts/pfbid0sFQgvdotD8uGY1hra82nqKrSsLhRKJsxAK2eYxTcHYsEFxEqT36h9dTm3iFJKC3Vl (visited on 07/10/2022).

[35] Mike Butcher. *As Iran Throttles Its Internet, Activists Fight to Get Online*. TechCrunch. 5th Oct. 2022. URL: https://techcrunch.com/2022/10/05/iran-internet-protests-censorship/ (visited on 16/10/2022).

[36] *How to Circumvent the Great Firewall and Connect to Tor from China? — Tor Project — Support*. URL: https://support.torproject.org/censorship/connecting-from-china/ (visited on 18/04/2023).

[37] *Tor Project — How Do Onion Services Work?* URL: https://community.torproject.org/onion-services/overview/ (visited on 19/04/2023).

[38] *Rend-Spec-v3.Txt - Torspec - Tor's Protocol Specifications*. URL: https://gitweb.torproject.org/torspec.git/tree/rend-spec-v3.txt (visited on 19/04/2023).

[39] *Attacked Over Tor - The Hacker Factor Blog*. URL: https://www.hackerfactor.com/blog/index.php?/archives/762-Attacked-Over-Tor.html (visited on 19/04/2023).

[40] *Tor Is Slow Right Now. Here Is What Is Happening. — Tor Project*. URL: https://blog.torproject.org/tor-network-ddos-attack/ (visited on 19/04/2023).

[41] *Dir-Spec.Txt - Tor's Protocol Specifications*. URL: https://gitweb.torproject.org/torspec.git/tree/dir-spec.txt (visited on 15/10/2022).

[42] *I2P: A Scalable Framework for Anonymous Communication - I2P*. URL: https://geti2p.net/en/docs/how/tech-intro (visited on 19/04/2023).

[43] *Guard-Spec.Txt - Torspec - Tor's Protocol Specifications*. URL: https://gitweb.torproject.org/torspec.git/tree/guard-spec.txt (visited on 23/10/2022).

[44] Nan Zhang et al. "gPath: A Game-Theoretic Path Selection Algorithm to Protect Tor's Anonymity". In: *Decision and Game Theory for Security*. Ed. by Tansu Alpcan, Levente Buttyán and John S. Baras. Berlin, Heidelberg: Springer, 2010, pp. 58–71. ISBN: 978-3-642-17197-0. DOI: 10.1007/978-3-642-17197-0_4.

[45] David L. Chaum. "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms". In: *Communications of the ACM* 24.2 (1st Feb. 1981), pp. 84–90. ISSN: 0001-0782. DOI: 10.1145/358549.358563.

[46] Roger Dingledine, Nick Mathewson and Paul Syverson. *Tor: The Second-Generation Onion Router:* Fort Belvoir, VA: Defense Technical Information Center, 1st Jan. 2004. DOI: 10.21236/ADA465464.

[47] Rob Jansen and Aaron Johnson. "Safely Measuring Tor". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS '16. New York, NY, USA: Association for Computing Machinery, 24th Oct. 2016, pp. 1553–1567. ISBN: 978-1-4503-4139-4. DOI: 10.1145/2976749.2978310.

[48] Cynthia Wagner et al. "Breaking Tor Anonymity with Game Theory and Data Mining". In: *Concurrency and Computation: Practice and Experience* 24.10 (2011), pp. 1052–1065. ISSN: 1532-0634. DOI: 10.1002/cpe.1828.

[49] *Gambit: Software Tools for Game Theory*. URL: http://www.gambit-project.org/ (visited on 29/10/2022).

[50] Milad Nasr and Amir Houmansadr. "GAME OF DECOYS: Optimal Decoy Routing Through Game Theory". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS '16. New York, NY, USA: Association for Computing Machinery, 24th Oct. 2016, pp. 1727–1738. ISBN: 978-1-4503-4139-4. DOI: 10.1145/2976749.2978367.

[51] Emmanouil Panaousis et al. "Game Theoretic Path Selection to Support Security in Device-to-Device Communications". In: *Ad Hoc Networks* 56 (1st Mar. 2017), pp. 28–42. ISSN: 1570-8705. DOI: 10.1016/j.adhoc.2016.11.008.

[52] Gruia Calinescu et al. "Adversary Games in Secure/Reliable Network Routing". In: *Game Theory for Networks*. Ed. by Rahul Jain and Rajgopal Kannan. Vol. 75. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 249–264. ISBN: 978-3-642-30372-2 978-3-642-30373-9. DOI: 10.1007/978-3-642-30373-9_18.

[53] Andriy Panchenko, Fabian Lanze and Thomas Engel. "Improving Performance and Anonymity in the Tor Network". In: *2012 IEEE 31st International Performance Computing and Communications Conference (IPCCC)*. 2012 IEEE 31st International Performance Computing and Communications Conference (IPCCC). Dec. 2012, pp. 1–10. DOI: 10.1109/PCCC.2012.6407715.

[54] Lauren Watson et al. "Privacy Preserving Detection of Path Bias Attacks in Tor." In: *Proc. Priv. Enhancing Technol.* 2020.4 (2020), pp. 111–130.

[55] *Path-Spec.Txt - Tor's Protocol Specifications*. URL: https://gitweb.torproject.org/torspec.git/tree/path-spec.txt (visited on 25/09/2022).

[56] Yixin Sun et al. "RAPTOR: Routing Attacks on Privacy in Tor". In: *Proceedings of the 24th USENIX Security Symposium* (12th Mar. 2015).

[57] Ye Zhu et al. "Correlation-Based Traffic Analysis Attacks on Anonymity Networks". In: *IEEE Transactions on Parallel and Distributed Systems* 21.7 (July 2010), pp. 954–967. ISSN: 1558-2183. DOI: 10.1109/TPDS.2009.146.

[58] Kevin Bauer et al. "Low-Resource Routing Attacks against Tor". In: *Proceedings of the 2007 ACM Workshop on Privacy in Electronic Society*. WPES '07. New York, NY, USA: Association for Computing Machinery, 29th Oct. 2007, pp. 11–20. ISBN: 978-1-59593-883-1. DOI: 10.1145/1314333.1314336.

[59] S.J. Murdoch and G. Danezis. "Low-Cost Traffic Analysis of Tor". In: *2005 IEEE Symposium on Security and Privacy (S&P'05)*. 2005 IEEE Symposium on Security and Privacy (S&P'05). May 2005, pp. 183–195. DOI: 10.1109/SP.2005.12.

[60] Rungrat Wiangsripanawan, Willy Susilo and Rei Safavi-Naini. "Design Principles for Low Latency Anonymous Network Systems Secure against Timing Attacks". In: *Proceedings of the Fifth Australasian Symposium on ACSW Frontiers - Volume 68*. ACSW '07. AUS: Australian Computer Society, Inc., 30th Jan. 2007, pp. 183–191.

[61] Milad Nasr, Alireza Bahramali and Amir Houmansadr. "DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning". In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. CCS '18. New York, NY, USA: Association for Computing Machinery, 15th Oct. 2018, pp. 1962–1976. ISBN: 978-1-4503-5693-0. DOI: 10.1145/3243734.3243824.

[62] *Tor Security Advisory: "Relay Early" Traffic Confirmation Attack — Tor Project*. URL: https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/ (visited on 30/10/2022).

[63] Noam Nisan et al. *Algorithmic Game Theory*. Cambridge University Press, 24th Sept. 2007. 776 pp. ISBN: 978-1-139-46654-7. Google Books: YCu2alSw0w8C.

[64] Martin J. Osborne. *An Introduction to Game Theory*. Oxford University Press, 2004. 533 pp. ISBN: 978-0-19-512896-3. Google Books: V2zAkgEACAAJ.

[65] Matthew O. Jackson. *A Brief Introduction to the Basics of Game Theory*. SSRN Scholarly Paper 1968579. Rochester, NY: Social Science Research Network, 5th Dec. 2011. DOI: 10.2139/ssrn.1968579.

[66] Ofcom. *UK Home Broadband Performance*. URL: https://www.ofcom.org.uk/__data/assets/pdf_file/0015/244140/home-broadband-report-2022.pdf.

[67] Elaine L May and Barbara A Zimmer. "The Evolutionary Development Model for Software". In: *Hewlett Packard Journal* 47 (1996), pp. 39–41.